



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**MAJOR PROJECT MID-TERM REPORT
ON
EXPLAINABLE WIND-AWARE SPATIO-TEMPORAL GRAPH NEURAL NETWORK
FOR AIR QUALITY FORECASTING**

Submitted By:

Aman Singh	(THA079BCT002)
Avinash Joshi	(THA079BCT007)
Kaushik Rayamajhi	(THA079BCT018)
Nirika Lamichhane	(THA079BCT024)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

July 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Institute of Engineering, Tribhuvan University, for including the major project as an important part of the Bachelor's Degree in Computer Engineering curriculum.

We are also thankful to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing academic support and guidance during the preparation of this mid-term report.

Finally, we would like to thank our project supervisor, teachers, friends, and contributors who directly or indirectly supported us during the first phase of this project.

Aman Singh	(THA079BCT002)
Avinash Joshi	(THA079BCT007)
Kaushik Rayamajhi	(THA079BCT018)
Nirika Lamichhane	(THA079BCT024)

ABSTRACT

Ambient air pollution is a major environmental and public-health concern in urban areas where pollutant concentration varies across space, time, and season. Kathmandu Valley is affected by road traffic, dust resuspension, construction activities, brick kilns, seasonal forest-fire episodes, valley topography, and changing meteorological conditions, making short-term $PM_{2.5}$ forecasting important for air quality interpretation and public awareness. Traditional station-wise forecasting models often ignore wind-driven pollutant movement between monitoring stations and usually provide numerical outputs without clearly explaining why a forecast is high or low. This mid-term report presents the progress of an explainable wind-aware Spatio-Temporal Graph Neural Network (ST-GNN) framework for short-term $PM_{2.5}$ forecasting, interpretation, and visualization in Kathmandu Valley. Monitoring stations are represented as graph nodes, while dynamic edges are designed using geographical distance, historical $PM_{2.5}$ correlation, wind speed, and wind-direction alignment. The primary forecasting model is a Graph Attention Network with Gated Recurrent Unit (GAT-GRU), where GAT captures spatial influence between stations and GRU captures temporal patterns from historical observations. Work completed during the first phase includes literature review, requirement analysis, system architecture design, dataset planning, preprocessing design, wind-aware graph formulation, and dashboard planning. The remaining phase will focus on model training, quantitative evaluation, explainability outputs, virtual-node validation, dashboard integration, and final testing. The project is intended as a research prototype and decision-support tool for air quality interpretation and visualization, not as a replacement for official air-quality monitoring or warning systems.

Keywords—*Air quality forecasting, $PM_{2.5}$, wind-aware graph, spatio-temporal graph neural network, GAT-GRU, explainable AI, AQI, Kathmandu Valley.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1. INTRODUCTION	1
1.1. Background	1
1.2. Motivation	2
1.3. Problem Definition	2
1.4. Objectives	3
1.5. Scope and Applications	3
2. LITERATURE REVIEW	4
2.1. Air Quality Forecasting as a Spatio-Temporal Problem	4
2.2. Graph Neural Networks for Air Quality Monitoring Networks	4
2.3. Spatio-Temporal GNNs for PM2.5 Forecasting	5
2.4. Wind-Aware Dynamic Graph Construction	5
2.5. GAT-GRU Based Forecasting	6
2.6. Virtual Nodes for Unmonitored Locations	7
2.7. Explainability in Air Quality Forecasting	7
2.8. Data Sources for PM2.5 and Meteorological Features	8
2.9. Kathmandu Valley Air Quality Context	8
2.10. Research Gap	9
3. REQUIREMENT ANALYSIS	10
3.1. Functional Requirements	10
3.2. Non-Functional Requirements	11
3.3. Project Requirements	11
3.4. Feasibility Study	12
4. SYSTEM ARCHITECTURE AND METHODOLOGY	13
4.1. System Block Diagram/Architecture	13
4.2. Description of Working Principle	14
4.3. Flow Diagram	24
4.4. Instrumentation Tools	25

4.5. Evaluation Methodology	25
5. IMPLEMENTATION DETAILS	27
5.1. Implementation Status	27
5.2. Data Preparation Module	27
5.3. Graph Construction Module	28
5.4. Forecasting Model Module	28
5.5. Virtual Node Module	28
5.6. Explainability Module	29
5.7. Dashboard Module	29
6. RESULTS AND ANALYSIS	30
6.1. Requirement and Design Results	30
6.2. Dataset Preparation Results	30
6.3. Architecture Analysis	31
6.4. Mid-Term Progress Analysis	31
6.5. Expected Evaluation Outputs	31
6.6. Discussion	32
7. REMAINING TASK	33
7.1. Completed Work	33
7.2. Pending Work	33
7.3. Risks and Mitigation	34
7.4. Remaining Milestones	34
A. PROJECT BUDGET	35
B. PROJECT TIMELINE	36
C. DATASET DETAILS	37
REFERENCES	38

LIST OF FIGURES

Figure 4-1. System Block Diagram	13
Figure 4-2. Spatial modeling using graph attention	21
Figure 4-3. Temporal modeling using GRU	22
Figure 4-4. Proposed air quality forecasting workflow	24
Figure B-1. Project Gantt Chart	36

LIST OF TABLES

Table 3-1. Hardware Requirements	11
Table 3-2. Software Requirements	12
Table 4-1. Illustrative Sample of the Integrated Dataset	15
Table 4-2. Proposed Technical Stack	25
Table 5-1. Mid-Term Implementation Status	27
Table 6-1. Model-Ready Dataset Structure	30
Table 6-2. Progress Analysis at Mid-Term Stage	31
Table 7-1. Remaining Project Milestones	34
Table A-1. Estimated Project Budget	35
Table C-1. Dataset Field Description	37

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AQI	Air Quality Index
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
PM	Particulate Matter
PM ₁	Particulate Matter with diameter less than 1 micrometer
PM _{2.5}	Particulate Matter with diameter less than 2.5 micrometers
REST	Representational State Transfer
RMSE	Root Mean Square Error
SHAP	Shapley Additive Explanations
ST-GNN	Spatio-Temporal Graph Neural Network
SVM	Support Vector Machine
UI	User Interface
XAI	Explainable Artificial Intelligence

1. INTRODUCTION

1.1. Background

Air pollution is one of the major environmental and public-health concerns in urban areas. Fine particulate matter, commonly known as $PM_{2.5}$, is especially important because it consists of very small airborne particles that can enter deep into the respiratory system and affect human health. $PM_{2.5}$ concentration is influenced by many factors such as vehicular emissions, road dust, construction activities, industrial sources, seasonal variation, temperature, humidity, wind speed, wind direction, and local geography.

Kathmandu Valley is a suitable study area for $PM_{2.5}$ forecasting because it experiences frequent air pollution problems due to dense urban activity, traffic emissions, construction dust, and its bowl-shaped valley structure. The surrounding hills and limited air circulation can contribute to pollutant accumulation under certain meteorological conditions. Therefore, short-term $PM_{2.5}$ forecasting in Kathmandu Valley can support public awareness, environmental monitoring, and air quality interpretation.

Air quality forecasting is a spatio-temporal problem. This means that pollution changes across both location and time. The future $PM_{2.5}$ concentration at one monitoring station may depend not only on its own previous readings, but also on nearby stations, wind movement, and meteorological conditions. Traditional time-series models can learn temporal patterns, but they may not fully capture spatial influence between monitoring stations.

Graph Neural Networks (GNNs) provide a suitable approach for this type of problem because air quality monitoring stations can be represented as a graph. In this graph, each monitoring station is treated as a node, and the relationship between stations is represented as an edge. By combining graph-based spatial learning with recurrent temporal learning, the system can model both station-to-station influence and time-based pollution variation.

In this project, an explainable wind-aware spatio-temporal graph neural network framework is proposed for $PM_{2.5}$ forecasting in Kathmandu Valley. The system will use historical $PM_{2.5}$ observations, meteorological variables, and station relationships to forecast $PM_{2.5}$ concentration after 6 hours. It will also provide explanation outputs to show which stations, environmental features, and previous time patterns influenced the prediction.

1.2. Motivation

Existing air quality forecasting approaches often focus mainly on individual monitoring stations or use fixed spatial relationships between stations. However, pollution does not remain fixed in one location. Wind can transport polluted air from one area to another, and the influence between stations can change depending on wind direction and wind speed. A fixed graph based only on distance may therefore fail to represent the changing nature of pollutant movement.

Kathmandu Valley also has limited monitoring coverage compared to the complexity of its urban environment. Some areas may not have physical air quality sensors, even though people living in those areas still need information about local air quality. This motivates the use of graph-based modeling and virtual nodes, where selected unmonitored map locations can be estimated using nearby real monitoring stations and wind-aware spatial relationships.

Another motivation is explainability. A forecasting system should not only produce a predicted $PM_{2.5}$ value; it should also provide understandable reasons behind the prediction. For example, the system should help explain whether a forecast was influenced by high $PM_{2.5}$ at a nearby station, wind flow from a polluted direction, low wind speed, humidity, or recent pollution trends. Such explanations can make the system more transparent and useful for environmental monitoring, interpretation, and public awareness.

1.3. Problem Definition

Short-term $PM_{2.5}$ forecasting in Kathmandu Valley is challenging because pollutant concentration is affected by both temporal and spatial factors. $PM_{2.5}$ at a particular station may depend on its previous values, meteorological conditions, wind direction, wind speed, and pollutant levels at nearby or upwind stations. Many conventional forecasting methods treat stations independently or use static spatial relationships, which may not properly represent changing pollutant transport caused by wind.

In addition, air quality monitoring stations are limited in number and cannot cover every road, neighborhood, or residential area. This creates difficulty in estimating $PM_{2.5}$ concentration at unmonitored locations. Many forecasting systems also provide only numerical predictions without explaining why the model made a certain forecast.

Therefore, the problem addressed in this project is to design and develop an explainable wind-aware spatio-temporal graph neural network model that can forecast $PM_{2.5}$ concentration after 6 hours for selected Kathmandu Valley monitoring stations and selected virtual locations. The system will model station relationships using distance,

PM_{2.5} correlation, wind direction, and wind speed, and will present interpretable outputs through a dashboard.

1.4. Objectives

The objectives of this project are:

1. To build a wind-aware dynamic graph from PM_{2.5} and meteorological data, incorporating distance, correlation, wind factors, and virtual nodes for spatial modeling.
2. To develop and evaluate GAT-GRU and baseline models for PM_{2.5} forecasting with an explainable dashboard showing forecasts, AQI, spatial patterns, virtual nodes, and model insights.

1.5. Scope and Applications

The scope of this project focuses on PM_{2.5} forecasting in Kathmandu Valley using selected monitoring stations from Kathmandu, Lalitpur, and Bhaktapur. The system will use historical PM_{2.5} observations and meteorological variables such as wind speed, wind direction, temperature, humidity, pressure, dew point, and temporal features where available.

The project will represent monitoring stations as graph nodes and model the relationship between stations using distance, PM_{2.5} correlation, and wind-aware features. In addition to physical monitoring stations, selected unmonitored map locations may be represented as virtual nodes to estimate PM_{2.5} levels in areas without direct sensor readings.

The system will be developed as a research prototype for forecasting, explanation, and visualization. The major applications include PM_{2.5} forecasting, station-level air quality interpretation, virtual-node-based estimation for unmonitored locations, pollution risk visualization, explainable environmental monitoring, and academic research support. The dashboard will help users understand predicted PM_{2.5} levels, Air Quality Index (AQI) categories, spatial pollution patterns, important influencing stations, and important environmental features behind each prediction.

2. LITERATURE REVIEW

2.1. Air Quality Forecasting as a Spatio-Temporal Problem

Air pollution forecasting is a challenging task because pollutant concentration changes across both space and time. Fine particulate matter, especially $PM_{2.5}$, is influenced by local emission sources, weather conditions, seasonal variation, and the movement of air between different locations. In an urban valley environment such as Kathmandu Valley, $PM_{2.5}$ levels may vary from one monitoring station to another depending on traffic density, construction activity, road dust, brick kiln emissions, wind movement, and local topography.

Traditional forecasting methods such as statistical models, regression-based models, support vector machines, random forest, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) have been widely used for air quality prediction. These methods are useful for learning historical patterns and short-term temporal trends. However, many of them treat each monitoring station independently. In reality, $PM_{2.5}$ concentration at one location may be affected by nearby or upwind locations. Therefore, air quality forecasting should consider both temporal dependency and spatial dependency.

Temporal dependency refers to how $PM_{2.5}$ changes over previous hours, while spatial dependency refers to how one monitoring station may influence another. This is especially important for Kathmandu Valley because wind and valley circulation can affect pollutant transport and accumulation. Therefore, a spatio-temporal modeling approach is suitable for this project.

2.2. Graph Neural Networks for Air Quality Monitoring Networks

GNNs provide an effective way to model relationships between air quality monitoring stations. In a graph-based representation, each monitoring station can be represented as a node, and the relationship between two stations can be represented as an edge. This allows the model to learn not only from the past readings of one station, but also from the behavior of connected stations.

Kipf and Welling proposed the Graph Convolutional Network, which learns node representations by aggregating information from neighboring nodes [1]. In air quality forecasting, this idea is useful because $PM_{2.5}$ at one station may be influenced by the pollution condition of nearby stations. A GCN-based model can therefore capture spatial relationships between monitoring locations.

However, all neighboring stations do not contribute equally. Some stations may have stronger influence because they are closer, have similar pollution patterns, or lie in the

direction of wind flow. Velickovic et al. proposed Graph Attention Networks, which allow the model to assign different attention weights to different neighboring nodes [2]. This is suitable for air quality forecasting because the model can learn which neighboring stations are more important for a particular prediction.

These graph-based methods provide the foundation for this project. In the proposed system, monitoring stations in Kathmandu Valley are represented as graph nodes, and the relationships between stations are modeled using distance, PM_{2.5} correlation, wind speed, and wind-direction alignment.

2.3. Spatio-Temporal GNNs for PM_{2.5} Forecasting

Several recent studies have shown that graph-based deep learning models can improve air quality forecasting by capturing both spatial and temporal patterns. Wang et al. proposed PM_{2.5}-GNN, a domain-knowledge enhanced graph neural network for PM_{2.5} forecasting [3]. Their work shows that PM_{2.5} prediction benefits from using graph-based relationships and environmental knowledge instead of relying only on individual station time-series data.

Pandey et al. proposed a spatial-diffusion guided encoder-decoder architecture for PM_{2.5} forecasting [4]. Their work highlights the importance of modeling the spread of PM_{2.5} across monitoring stations. This is closely related to the proposed project because PM_{2.5} movement in Kathmandu Valley may also depend on spatial relationships and weather-driven transport.

Panja et al. proposed E-STGCN, an extreme spatio-temporal graph convolutional network for air quality forecasting [5]. Their study is useful because it shows how spatio-temporal graph models can be applied to pollution forecasting tasks where high-pollution events are important. In this project, peak-event evaluation will be used to examine whether the model can identify periods of elevated PM_{2.5} concentration.

These studies support the use of spatio-temporal graph neural networks for PM_{2.5} forecasting. However, many forecasting models still depend on fixed graph structures. A fixed graph may use only distance or historical correlation, but it may not fully represent changing pollutant movement caused by wind. This creates the need for a wind-aware dynamic graph.

2.4. Wind-Aware Dynamic Graph Construction

Pollutant movement is affected by wind direction and wind speed. If wind flows from one monitoring station toward another, the first station may have a stronger influence on the second station. If the wind direction changes, the relationship between stations

may also change. Therefore, using a static graph for air quality forecasting may not be sufficient.

HighAir, proposed by Chen et al., uses a hierarchical graph neural network for air quality forecasting and considers wind direction in the graph structure [6]. This supports the idea that station relationships should not be based only on distance, but should also consider meteorological movement.

Xu et al. proposed a Dynamic Graph Neural Network with Adaptive Edge Attributes for air quality prediction [7]. Their work shows that dynamic edge attributes can better represent changing relationships between monitoring sites. This is important for the proposed system because the graph will be updated using distance, $PM_{2.5}$ correlation, wind direction, and wind speed.

AirPhyNet, proposed by Hettige et al., introduces a physics-guided approach for air quality prediction [8]. It supports the idea that air pollution modeling should consider physical processes such as diffusion and wind-driven transport. Although this project does not attempt to build a full physics-based atmospheric model, the proposed wind-aware graph follows the same practical idea that pollutant movement is affected by meteorological conditions.

Based on these studies, the proposed project uses a wind-aware dynamic graph for Kathmandu Valley. The graph is designed to represent changing station-to-station influence instead of assuming that all spatial relationships remain fixed over time.

2.5. GAT-GRU Based Forecasting

The proposed forecasting model combines a Graph Attention Network and a Gated Recurrent Unit. The GAT component captures spatial influence between monitoring stations by learning attention weights among connected nodes. This helps the model identify which neighboring stations are more important for a target station at a given time.

The GRU component captures temporal dependency from historical $PM_{2.5}$ and meteorological observations. GRU is suitable for time-series forecasting because it can learn patterns from previous time steps while being simpler and computationally lighter than some other recurrent architectures.

The combination of GAT and GRU is suitable for this project because $PM_{2.5}$ forecasting in Kathmandu Valley requires both spatial and temporal learning. The GAT layer models station-to-station influence, while the GRU layer learns how pollution evolves over

time. The output of the model is a future $PM_{2.5}$ concentration value for each selected monitoring station and virtual node.

2.6. Virtual Nodes for Unmonitored Locations

A practical limitation of air quality monitoring is that sensors are available only at selected locations. Kathmandu Valley contains many roads, residential areas, and urban zones where direct $PM_{2.5}$ monitoring may not be available. Because of this, station-level forecasting alone may not provide enough spatial coverage.

To address this problem, the proposed project includes virtual nodes. A virtual node represents an unmonitored map location. It does not have direct sensor readings, so its features and connections are estimated using nearby real monitoring stations, distance, wind direction, wind speed, and pollution similarity. This allows the graph model to estimate $PM_{2.5}$ concentration at selected locations without physical sensors.

The virtual-node approach improves the practical usability of the dashboard because it can help generate a more continuous view of air quality patterns across Kathmandu Valley. The approach can be evaluated using a leave-one-station-out method, where one real monitoring station is temporarily treated as a virtual node and the predicted $PM_{2.5}$ value is compared with the actual reading.

2.7. Explainability in Air Quality Forecasting

Deep learning models often provide accurate predictions, but they may not clearly explain why a prediction was made. In air quality forecasting, explanation is important because users need to understand the possible reasons behind high or low predicted $PM_{2.5}$ levels.

Ying et al. proposed GNNExplainer, which provides explanations for graph neural network predictions by identifying important graph structures and node features [9]. This idea is useful for the proposed system because it can help identify which stations had the greatest influence on a forecast.

Lundberg and Lee proposed SHAP, a widely used method for explaining machine-learning predictions through feature contribution values [10]. SHAP can help show how features such as past $PM_{2.5}$, wind speed, wind direction, humidity, temperature, and temporal variables contribute to the final prediction.

In this project, explainability will be provided through attention-based station influence, feature attribution, and temporal sensitivity analysis. This means the system will not only forecast $PM_{2.5}$ values but also help users understand the major factors behind the forecast.

2.8. Data Sources for PM_{2.5} and Meteorological Features

The proposed system requires both air quality data and meteorological data. OpenAQ provides access to air quality measurements from monitoring stations and can be used to collect PM_{2.5} observations where station data is available [11]. These PM_{2.5} records will serve as the main target variable for forecasting.

Meteorological data is also necessary because weather conditions strongly affect pollutant concentration and transport. Open-Meteo provides historical weather variables such as temperature, humidity, pressure, wind speed, wind direction, and dew point [12]. In this project, wind speed and wind direction are especially important because they are used in the wind-aware graph construction process.

By combining PM_{2.5} observations with meteorological features, the proposed model can learn both pollution trends and weather-related influence on air quality.

2.9. Kathmandu Valley Air Quality Context

Kathmandu Valley is the selected study area for this project. The valley is an important case for PM_{2.5} forecasting because it experiences frequent air pollution problems due to urban emissions, construction dust, road dust, brick kilns, seasonal forest-fire episodes, and valley topography. The bowl-shaped geography of the valley can limit air dispersion under certain conditions, making pollutant accumulation an important local concern.

Becker et al. studied particulate matter variability in Kathmandu using in-situ measurements, remote sensing, and reanalysis data [13]. Their work is useful for this project because it shows that particulate matter pollution in Kathmandu has strong temporal and seasonal variation. It also supports the use of external meteorological and reanalysis data for understanding local air pollution behavior.

Panday and Prinn studied the diurnal cycle of air pollution in Kathmandu Valley [14]. Their work is important because it shows that pollution in the valley changes throughout the day and is influenced by local meteorology and valley circulation. This supports the need for temporal modeling and wind-aware analysis in Kathmandu Valley.

Clean Energy Nepal highlights that Kathmandu Valley is vulnerable to air pollution due to rapid urbanization, emission sources, valley topography, and meteorological conditions [15]. This supports the motivation for developing a forecasting and visualization system focused on Kathmandu Valley.

ICIMOD reports the importance of air quality management efforts in Kathmandu Valley and emphasizes the need for better information systems and decision-support

mechanisms [16]. This supports the practical relevance of developing a dashboard-based research prototype for air quality forecasting, explanation, and visualization.

2.10. Research Gap

The reviewed literature shows that graph neural networks and spatio-temporal deep learning models are effective for air quality forecasting. Existing studies have shown that GNNs can model relationships between monitoring stations, attention mechanisms can learn unequal station influence, and recurrent models can capture temporal pollution patterns. Studies such as PM_{2.5}-GNN, HighAir, dynamic graph neural networks with adaptive edge attributes, and AirPhyNet provide strong technical support for graph-based, wind-aware, and physically meaningful air quality forecasting.

However, some gaps still remain. Many forecasting systems use fixed spatial relationships even though pollutant movement changes with wind direction and wind speed. Many models focus mainly on numerical prediction accuracy and provide limited explanation of why a forecast is high or low. In addition, monitoring stations cannot cover every location, so there is a need to estimate pollution at selected unmonitored areas using graph-based methods.

For Kathmandu Valley, previous studies have examined particulate matter variability, diurnal pollution patterns, and local air quality challenges. However, there is still a need for an integrated forecasting framework that combines wind-aware dynamic graph construction, GAT-GRU based PM_{2.5} forecasting, virtual-node estimation, explainability, and dashboard visualization for Kathmandu Valley.

This project addresses that gap by developing an explainable wind-aware spatio-temporal graph neural network framework for PM_{2.5} forecasting in Kathmandu Valley. The system represents monitoring stations as graph nodes, updates station relationships using distance, PM_{2.5} correlation, wind direction, and wind speed, predicts PM_{2.5} at a 6-hour horizon, estimates selected unmonitored locations through virtual nodes, and presents the results through an interactive dashboard.

3. REQUIREMENT ANALYSIS

This chapter describes the functional, non-functional, hardware, software, and feasibility requirements of the proposed explainable wind-aware spatio-temporal graph neural network system. Since the project is a software-based research prototype, the major requirements are related to data availability, model development, evaluation, explainability, and dashboard visualization.

3.1. Functional Requirements

The system must support the complete workflow from data preparation to visualization. The major functional requirements are listed below:

1. **FR-001: Data collection and integration:** The system shall collect or import PM_{2.5} observations from selected Kathmandu Valley monitoring stations and combine them with meteorological variables such as wind speed, wind direction, temperature, humidity, pressure, and dew point.
2. **FR-002: Data preprocessing:** The system shall clean missing values, align station records by timestamp, remove invalid observations, normalize numerical features, and generate model-ready temporal windows.
3. **FR-003: Wind-aware graph construction:** The system shall represent air quality stations as graph nodes and create dynamic edges using geographical distance, PM_{2.5} correlation, wind speed, and wind-direction alignment.
4. **FR-004: Forecasting:** The system shall forecast future PM_{2.5} concentration for selected monitoring stations at a primary horizon of 6 hours, with additional horizons used for comparison where possible.
5. **FR-005: Baseline comparison:** The system shall compare the proposed GAT-GRU model with baseline models such as persistence, regression-based models, and GRU.
6. **FR-006: Virtual-node estimation:** The system shall estimate PM_{2.5} concentration at selected unmonitored locations by representing them as virtual nodes in the graph.
7. **FR-007: Explainability:** The system shall provide interpretable outputs such as station influence, feature importance, and temporal importance to explain model predictions.
8. **FR-008: Dashboard visualization:** The system shall display forecasts, AQI categories, station-wise trends, spatial maps, virtual-node outputs, and explanation charts through a web dashboard.

3.2. Non-Functional Requirements

The non-functional requirements define how well the system should perform and how it should behave during use.

1. **NFR-001 (Accuracy):** The forecasting model should achieve lower MAE and RMSE than simple baseline models on the selected test dataset.
2. **NFR-002 (Usability):** The dashboard should present forecasts, maps, and explanations in a clear form suitable for academic demonstration and air quality interpretation.
3. **NFR-003 (Maintainability):** The codebase should be modular, separating data preprocessing, graph construction, model training, evaluation, explanation, backend service, and frontend visualization.
4. **NFR-004 (Scalability):** The system should allow additional stations, features, and forecasting horizons to be added without major redesign.
5. **NFR-005 (Reproducibility):** Experiments should use documented preprocessing steps, saved configuration values, and consistent train-validation-test splits.
6. **NFR-006 (Performance):** The trained model should generate forecasts fast enough for dashboard demonstration on a local or modest cloud environment.

3.3. Project Requirements

3.3.1. Hardware Requirements

The project does not require special sensing hardware because it uses available air quality and meteorological datasets. The development and experimentation requirements are listed in Table 3-1.

Table 3-1. Hardware Requirements

Component	Requirement
Processor	64-bit multi-core processor for preprocessing, training, and dashboard development
Memory	Minimum 8 GB RAM; 16 GB or higher recommended for model experimentation
Storage	At least 10 GB free storage for datasets, trained models, reports, and experiment logs
GPU	Optional GPU through local hardware, Google Colab, Kaggle, or another cloud environment for faster training
Internet	Required for dataset access, API use, dependency installation, and research review

3.3.2. Software Requirements

The software requirements are listed in Table 3-2. These tools support data processing, graph modeling, deep learning, explainability, API development, and dashboard visualization.

Table 3-2. Software Requirements

Category	Tools and Libraries
Programming language	Python 3.x, JavaScript or TypeScript
Data processing	pandas, NumPy, scikit-learn
Deep learning	PyTorch, PyTorch Geometric
Explainability	SHAP, attention-weight visualization, feature-importance analysis
Data sources	OpenAQ API or downloaded OpenAQ data, Open-Meteo historical weather API
Backend	FastAPI and REST API endpoints
Frontend	React, Leaflet or Mapbox, charting library such as Recharts
Documentation	LaTeX, BibTeX, Git

3.4. Feasibility Study

3.4.1. Technical Feasibility

The project is technically feasible because the required datasets, algorithms, and tools are available. PM_{2.5} records can be obtained from air quality monitoring data sources, while meteorological features can be obtained from Open-Meteo. The forecasting model can be implemented using established graph neural network and recurrent neural network libraries.

3.4.2. Economic Feasibility

The project is economically feasible because most required tools are free and open source. The major costs are related to internet access, optional cloud compute, printing, and report binding. A detailed project budget is included in Appendix A.

3.4.3. Schedule Feasibility

The project can be completed within the academic timeline by following a phased workflow. Literature review, system design, requirement analysis, and graph formulation have been completed during the first phase. Model implementation, result evaluation, explainability, dashboard integration, and final documentation remain for the next phase. The project timeline is included in Appendix B.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. System Block Diagram/Architecture

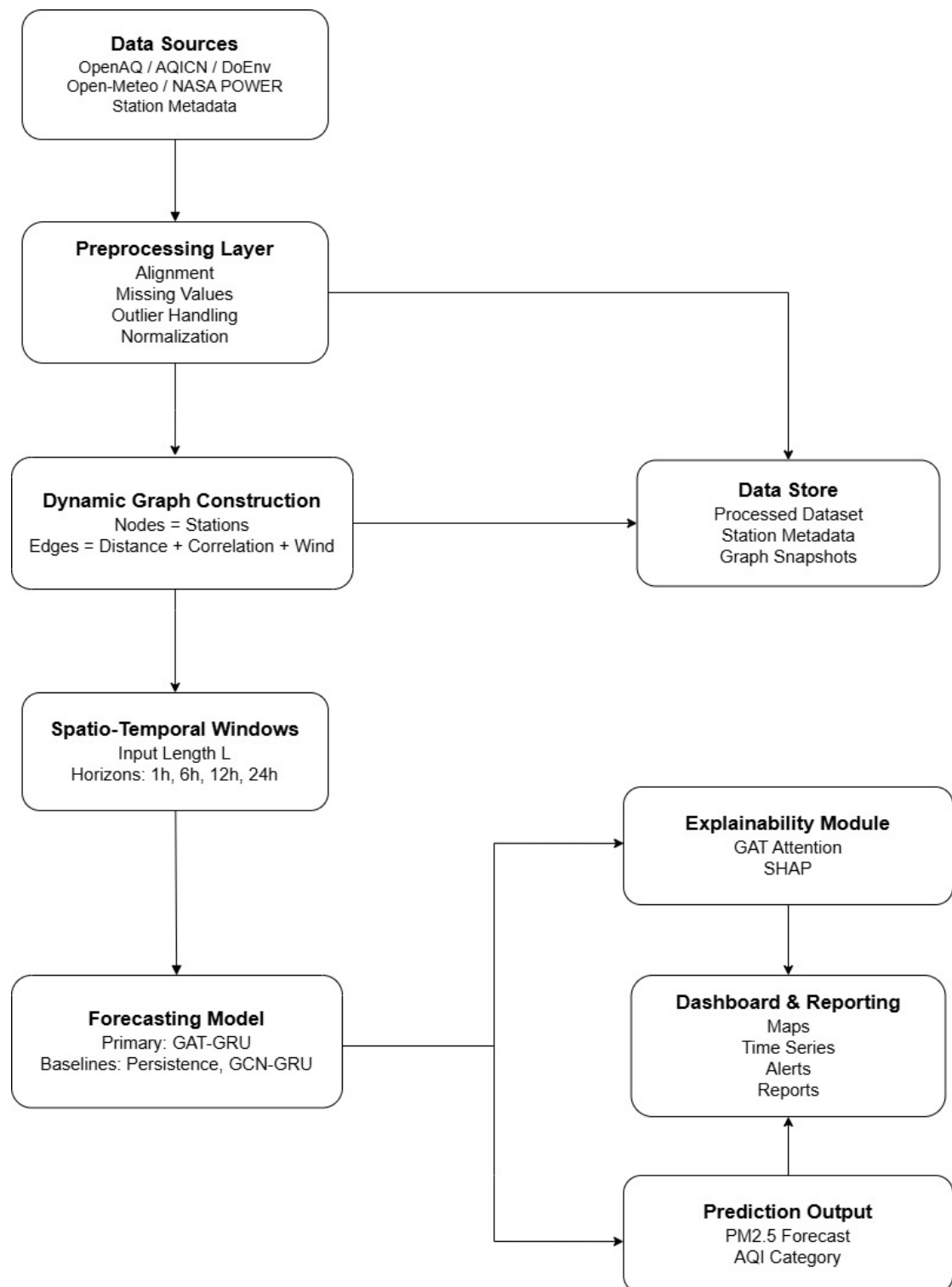


Figure 4-1. System Block Diagram

4.2. Description of Working Principle

The proposed system works as a wind-aware air quality forecasting and decision-support system for short-term $PM_{2.5}$ prediction. It combines air quality data, meteorological data, dynamic graph construction, spatio-temporal deep learning, explainability, and dashboard-based visualization. The overall working principle of the system is described in the following subsections.

4.2.1. Study Area, Stations and Data Access Status

The initial study area for this project will be Kathmandu Valley, Nepal, covering major urban and semi-urban areas of Kathmandu, Lalitpur, and Bhaktapur. Kathmandu Valley is selected as the primary study area because it experiences high air pollution due to traffic emissions, construction activities, industrial sources, dust, seasonal variation, and its bowl-shaped geography, which can trap pollutants.

In the first stage of the project, the study will focus on selected air quality monitoring stations within Kathmandu Valley. These stations will be used to collect historical pollutant data such as $PM_{2.5}$ and PM_{10} , along with meteorological data such as wind speed, wind direction, temperature, humidity, and pressure where available. The initially considered stations include Ratnapark, Shankapark, TU Kirtipur, Khumaltar, Bhaishipati, Pulchowk, Bhaktapur/Birendra School, and the US Embassy monitoring station. These stations are selected because they provide coverage across different parts of the valley, including dense urban areas, residential zones, and surrounding valley regions.

For model development, each monitoring station will be represented as a node in a graph. The relationship between stations will be defined using factors such as geographical distance, wind direction, wind speed, and pollutant similarity.

4.2.2. Dataset Description

The currently downloaded OpenAQ station datasets include $PM_{2.5}$, PM_{10} , relative humidity, and temperature. $PM_{2.5}$ concentration will serve as the target variable, whereas the remaining available variables will be used as input features. Wind speed, wind direction, atmospheric pressure, and dew point obtained from the Open-Meteo API will be considered as additional meteorological features. Wind speed and wind direction are particularly important for modeling the transport of airborne pollutants between monitoring stations. The spatial graph will therefore account for both geographical proximity and time-varying meteorological conditions.

Table 4-1 presents the proposed structure of the integrated dataset. The values are illustrative examples included only to demonstrate the organization of the model-ready records.

Table 4-1. Illustrative Sample of the Integrated Dataset

Timestamp	Station	PM _{2.5}	PM ₁	Wind Speed	Other Features
2026-01-01 01:00	Ratnapark	72.4	41.2	1.8	RH, temperature, pressure, dew point, wind direction
2026-01-01 01:00	Khumaltar	58.7	33.5	2.1	RH, temperature, pressure, dew point, wind direction
2026-01-01 01:00	Bhaktapur	64.9	37.8	1.5	RH, temperature, pressure, dew point, wind direction

In Table 4-1, PM_{2.5} and PM₁ concentrations are expressed in $\mu\text{g}/\text{m}^3$, relative humidity is expressed as a percentage, temperature and dew point are expressed in degrees Celsius, atmospheric pressure is expressed in hPa, wind speed is expressed in m/s, and wind direction is expressed in degrees. The displayed values are illustrative rather than actual observations.

4.2.3. Input Variables, Resolution and Prediction Target

The proposed model will use hourly air quality and meteorological observations from multiple monitoring stations. Let N represent the number of monitoring stations and F represent the number of input features. At each time step t , the observations are represented as a node feature matrix:

$$X_t \in \mathbb{R}^{N \times F} \quad (4.1)$$

where each row represents a monitoring station and each column represents a pollutant, meteorological, or temporal feature.

To capture temporal patterns, the model will use a sliding window of past observations. The default input window will be 24 hours, with a 48-hour window also tested for comparison. The primary forecasting horizon is 6 hours. Additional horizons of 1, 12, and 24 hours will be tested for comparison.

The main input variables will include $PM_{2.5}$, PM_1 , temperature, humidity, wind speed, wind direction, and temporal features such as hour of day, day of week, month, and season. Wind direction will be converted into sine and cosine components because it is a circular variable. Missingness indicators may also be added to show whether a value was observed or imputed.

The primary prediction target is future $PM_{2.5}$ concentration at each monitoring station. For station i and forecasting horizon h , the target value is represented as:

$$y_i^{(t+h)} \quad (4.2)$$

and the model prediction is represented as:

$$\hat{y}_i^{(t+h)} \quad (4.3)$$

Therefore, the task is formulated as a regression problem, where the model learns to minimize the difference between actual and predicted $PM_{2.5}$ values across all stations and time steps. Although the model predicts numerical $PM_{2.5}$ values, air quality categories such as good, moderate, unhealthy, and hazardous will be generated later in the post-processing stage using standard AQI threshold values.

4.2.4. Data Preprocessing

The collected air quality and meteorological data undergo a preprocessing stage before being used for graph construction and model training. Since the data are obtained from multiple sources, including OpenAQ and Open-Meteo, preprocessing is necessary to ensure consistency, reliability, and compatibility across all monitoring stations and environmental variables.

Air quality observations and meteorological records are first aligned according to their timestamps. Since the proposed system operates on hourly observations, all records are resampled and synchronized to a common hourly interval. For each monitoring station, $PM_{2.5}$, PM_1 , temperature, humidity, wind speed, and wind direction measurements corresponding to the same timestamp are merged into a unified observation record. This temporal alignment ensures that all input variables represent the same environmental conditions at a given time step and prevents inconsistencies during model training.

The collected datasets are inspected for missing observations. Any missing values identified within the pollutant or meteorological variables are handled using appropriate interpolation techniques. Short gaps in continuous measurements are filled through linear

interpolation, while larger missing segments may be removed if sufficient neighboring information is unavailable. This process helps preserve temporal continuity while reducing information loss caused by incomplete observations.

Environmental monitoring data may contain abnormal values caused by sensor malfunctions, communication errors, or measurement noise. Therefore, the dataset is examined for unrealistic or extreme observations. Basic statistical analysis and visualization techniques are used to identify potential outliers. Records containing physically impossible values, such as negative pollutant concentrations or invalid meteorological measurements, are removed or corrected according to data quality guidelines using either capping or log transformation.

The collected variables have different numerical ranges and units. To ensure stable neural network training, all numerical features are normalized before being supplied to the forecasting model. Min-Max normalization is applied to transform each feature into a common numerical range between 0 and 1. The normalized value of each feature is computed as:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (4.4)$$

where X represents the original feature value, X_{min} is the minimum observed value, and X_{max} is the maximum observed value within the training dataset.

After preprocessing and normalization, the sequential observations are converted into fixed-length temporal windows for forecasting. A sliding-window approach is used to construct input sequences from historical observations. For example, if a 24-hour look-back period is selected, the model receives pollutant and meteorological observations from the previous 24 hours and predicts the $PM_{2.5}$ and PM_{10} concentration for the next forecasting horizon.

The final output of the preprocessing stage is a clean, synchronized, normalized, and temporally structured dataset containing station-wise pollutant and meteorological observations. These processed records are subsequently used for dynamic graph construction and spatio-temporal forecasting.

4.2.5. Wind-Aware Dynamic Graph Construction

The proposed system models air quality monitoring stations as a dynamic graph in order to capture spatial and meteorological dependencies among different locations within Kathmandu Valley. Unlike conventional static graphs that use only fixed geographical distance or historical correlation, the proposed approach constructs a wind-aware

dynamic graph that adapts edge relationships based on time-varying meteorological conditions.

Let the air quality monitoring network be defined as a graph:

$$G_t = (V, E_t, A_t) \quad (4.5)$$

where V represents the set of monitoring stations, E_t represents the set of edges at time t , and A_t represents the adjacency matrix at time t . Each node $v_i \in V$ corresponds to a monitoring station and is associated with a feature vector:

$$X_i^t = [PM_{2.5}, PM_1, Temperature, Humidity, WindSpeed, WindDirection] \quad (4.6)$$

An edge between two stations i and j is defined based on geographical distance, historical pollutant correlation, wind speed, and wind direction alignment. Thus, the edge weight is defined as a composite function:

$$A_{ij}^t = f_d(i, j) \cdot f_c(i, j) \cdot f_w(i, j, t) \quad (4.7)$$

Spatial proximity between stations is modeled using normalized inverse distance:

$$f_d(i, j) = \exp\left(-\frac{d_{ij}}{\sigma_d}\right) \quad (4.8)$$

where d_{ij} is the geographical distance between stations i and j , and σ_d is a scaling parameter controlling spatial decay.

To capture long-term similarity in pollution patterns, historical correlation is computed as:

$$f_c(i, j) = \text{corr}(PM_{2.5_i}, PM_{2.5_j}) \quad (4.9)$$

The key contribution of the proposed system is the incorporation of wind-driven pollutant transport. Wind influence is modeled as:

$$f_w(i, j, t) = \alpha \cdot \cos(\theta_{ij} - \theta_t) \cdot WS_t \quad (4.10)$$

where θ_{ij} is the direction from station i to station j , θ_t is the wind direction at time t , WS_t is the wind speed at time t , and α is a scaling factor. This formulation increases influence when wind flows from one station toward another, decreases influence when wind flows in the opposite direction, and amplifies spatial propagation effects during stronger wind conditions.

The final adjacency matrix is time-dependent and is updated at every time step to reflect changing meteorological conditions:

$$A_t = \begin{bmatrix} A_{11}^t & A_{12}^t & \cdots & A_{1n}^t \\ A_{21}^t & A_{22}^t & \cdots & A_{2n}^t \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1}^t & A_{n2}^t & \cdots & A_{nn}^t \end{bmatrix} \quad (4.11)$$

The output of this module is the dynamic adjacency matrix A_t , the node feature matrix X_t , and the edge-weighted graph representation G_t . These outputs are subsequently used as input to the graph attention layer for spatial feature learning.

4.2.6. Virtual Node Construction for Unmonitored Locations

A key novelty of this project is the use of virtual nodes to estimate air pollution at locations where physical air quality monitoring stations are not available. In real-world settings, monitoring stations are limited and cannot represent every road, neighborhood, or residential area. To address this limitation, the proposed system will allow selected map locations to be represented as virtual nodes within the graph.

A virtual node represents an unmonitored geographic location selected from the map. Unlike physical stations, this node does not have direct sensor readings. Therefore, its initial feature values will be estimated using information from nearby monitoring stations. The connection between the virtual node and real stations will be determined using geographical distance, wind direction, wind speed, and pollutant similarity. Stations that are closer or located along the wind flow direction will be assigned stronger influence.

Formally, if there are N real monitoring stations and V virtual nodes, the graph is expanded as:

$$N' = N + V \quad (4.12)$$

where N' is the total number of real and virtual nodes in the graph. The model then performs prediction over both physical monitoring stations and virtual nodes.

The virtual node mechanism enables the system to estimate $\text{PM}_{2.5}$ concentration at unmonitored locations and generate a more continuous air pollution map. This is useful for identifying pollution patterns in areas without sensors and for improving the practical usability of the forecasting system.

To validate this approach, a leave-one-station-out strategy may be used. In this method, one real station is temporarily treated as a virtual node, and its actual $\text{PM}_{2.5}$ values are hidden from the model during prediction. The predicted values are then compared with the actual station readings. This allows the performance of virtual-node-based prediction to be evaluated even when real ground truth is not available for completely unmonitored locations.

4.2.7. Spatio-Temporal Forecasting Model

The proposed system employs a hybrid Graph Attention Network and Gated Recurrent Unit architecture to model both spatial and temporal dependencies in air quality data. This combination enables the model to capture pollutant interactions across monitoring stations as well as temporal variations over time.

In addition to forecasting $\text{PM}_{2.5}$ at physical monitoring stations, the model will also support prediction at virtual nodes. These virtual nodes represent unmonitored map locations and are connected to the existing station graph using wind-aware spatial relationships. This allows the GAT-GRU model to estimate pollution levels beyond the available sensor network.

At each time step t , the model receives a node feature matrix:

$$X_t \in \mathbb{R}^{N \times F} \quad (4.13)$$

and a dynamic adjacency matrix:

$$A_t \in \mathbb{R}^{N \times N} \quad (4.14)$$

where N represents the number of monitoring stations and F represents the number of input features per station. The adjacency matrix is generated from the wind-aware graph using factors such as geographical distance, wind direction, wind speed, and pollutant similarity.

The Graph Attention Network captures spatial influence between monitoring stations. For each station i , the GAT layer aggregates information from neighboring stations using attention weights:

$$h_i^t = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^t W x_j^t \right) \quad (4.15)$$

where x_j^t is the feature vector of neighboring station j , W is a learnable weight matrix, α_{ij}^t is the attention coefficient, $\mathcal{N}(i)$ represents the neighboring stations of station i , and σ is a non-linear activation function. The attention coefficient α_{ij}^t shows how much influence station j has on station i at time t . This also supports model explainability because the learned attention weights can help identify which nearby stations contribute most to a prediction.

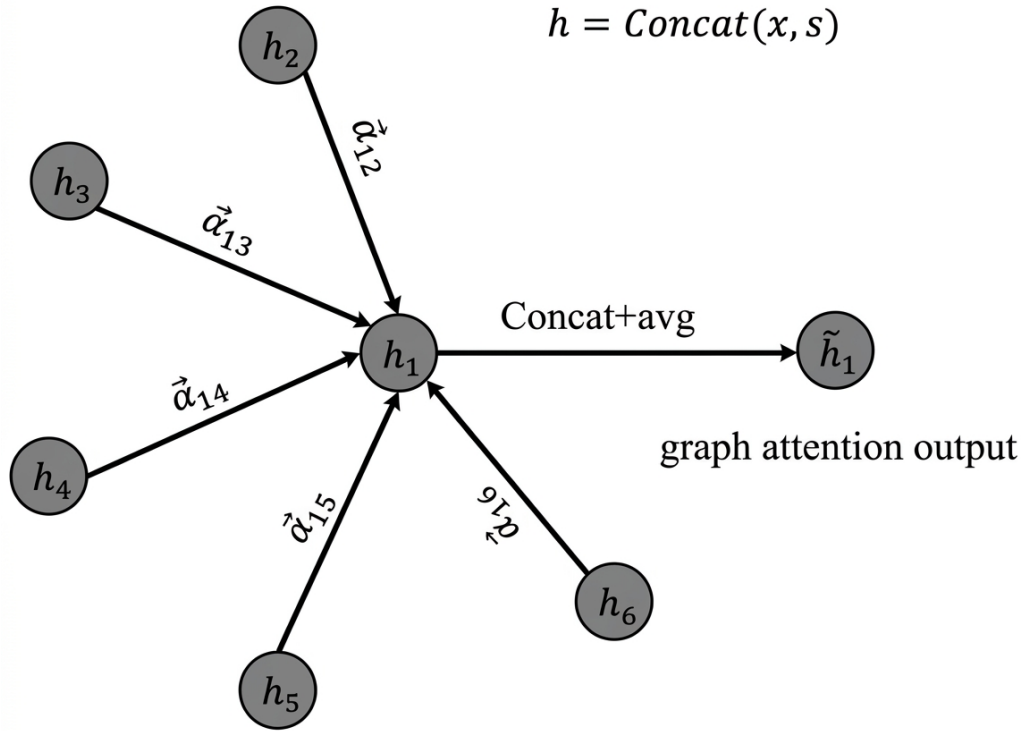


Figure 4-2. Spatial modeling using graph attention

The spatial output of the GAT layer is represented as:

$$H_t = GAT(X_t, A_t) \quad (4.16)$$

To capture temporal dependencies, the spatial embeddings from multiple previous time steps are passed into a GRU network. For an input window of length L , the GRU processes:

$$H_{t-L+1:t} \quad (4.17)$$

The hidden state is updated as:

$$z_t = GRU(H_t, z_{t-1}) \quad (4.18)$$

where H_t is the spatial embedding at time t , and z_t is the hidden state that captures temporal evolution. This allows the model to learn short-term pollution changes, delayed meteorological effects, and temporal persistence in $PM_{2.5}$ concentration.

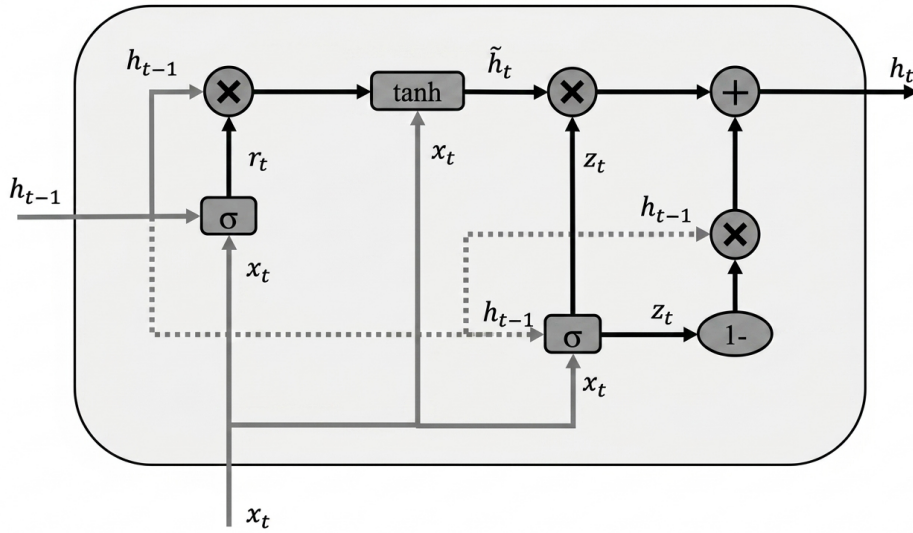


Figure 4-3. Temporal modeling using GRU

The final hidden representation from the GRU is passed through a fully connected regression layer to predict future $PM_{2.5}$ concentration at each station:

$$\hat{y}_{t+h} = W_o z_t + b \quad (4.19)$$

where $\hat{y}_{t+h}$ is the predicted $PM_{2.5}$ concentration after horizon h , and $h \in \{1, 6, 12, 24\}$. The one-hour horizon represents the single-step forecasting case, while 6, 12, and 24-hour horizons are used for multi-horizon forecasting. The overall architecture can be

summarized as input features, wind-aware graph construction, GAT layer, GRU layer, and regression output.

4.2.8. Explainability Module

The proposed system includes an explainability module to help interpret the predictions generated by the wind-aware spatio-temporal forecasting model. Since the model uses GAT and GRU layers, the explanation focuses on spatial influence, feature contribution, and temporal importance.

Spatial explainability is obtained from the attention weights learned by the GAT layer. For a target station or virtual node, the attention coefficient α_{ij} shows how much neighboring station j contributes to the prediction at node i . These attention values help identify the most influential stations, possible pollutant movement direction, and the effect of wind-aware graph connections. This is especially useful when estimating $\text{PM}_{2.5}$ at virtual nodes, as it can show which real monitoring stations influenced the predicted value.

Feature-level explainability will be analyzed using SHAP or similar feature importance methods. This will help identify how much each input variable, such as past $\text{PM}_{2.5}$, PM_{10} , wind speed, wind direction, temperature, humidity, and temporal features, contributes to the final prediction. This allows the system to explain why a predicted pollution value is high or low.

Temporal interpretability will be studied by analyzing how different historical time windows affect the forecast. Since the GRU learns from past observations, this analysis helps understand whether the prediction is mainly influenced by recent pollution spikes, delayed weather effects, or persistent pollution patterns.

The explainability outputs will be presented through visualizations such as station influence maps, attention-based graph plots, feature importance charts, and time-series plots. These explanations will make the model more transparent and useful for air quality monitoring, pollution interpretation, and decision support.

4.3. Flow Diagram

Figure 4-4 presents the overall workflow from data collection to dashboard visualization.

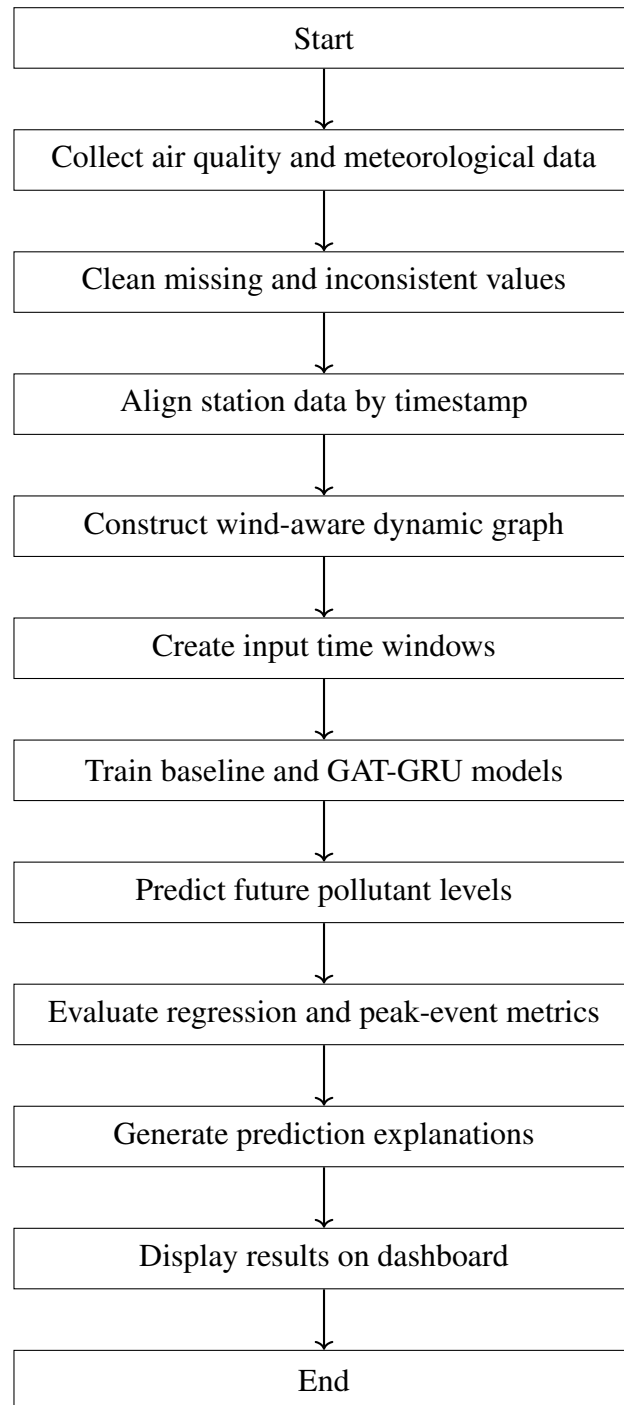


Figure 4-4. Proposed air quality forecasting workflow

4.4. Instrumentation Tools

Table 4-2. Proposed Technical Stack

Category	Tools and Technologies
Data Processing	Python, pandas, NumPy, scikit-learn
Deep Learning	PyTorch, PyTorch Geometric, SHAP
Forecasting Models	Persistence model, linear/tree regression, GRU, GAT-GRU
Data Sources	OpenAQ, Open-Meteo
Backend	FastAPI, Representational State Transfer (REST) services
Frontend	React, Leaflet or Mapbox, Recharts
Database and Storage	PostgreSQL, CSV/Parquet files, local storage for datasets, models, and experiment logs
Deployment	Docker, local server, optional cloud deployment

4.5. Evaluation Methodology

4.5.1. Evaluation Metrics

The project will evaluate forecasting models using regression metrics and horizon-wise error analysis. The proposed GAT-GRU model will be compared with baseline models to check whether dynamic graph learning improves forecasting accuracy.

Mean Absolute Error (MAE) measures the average absolute prediction error in $PM_{2.5}$ values:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.20)$$

Root Mean Squared Error (RMSE) penalizes large errors more strongly and is useful for evaluating pollution spikes:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.21)$$

Mean Absolute Percentage Error (MAPE) measures prediction error in percentage form:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100 \quad (4.22)$$

The R^2 score shows how well the model explains variation in $PM_{2.5}$ concentration:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (4.23)$$

Bias indicates whether the model generally overpredicts or underpredicts:

$$Bias = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \quad (4.24)$$

4.5.2. Peak Metrics

Peak-event metrics will be used to evaluate whether the system can identify high-pollution events. Precision measures how many predicted pollution events were actually correct:

$$Precision = \frac{TP}{TP + FP} \quad (4.25)$$

Recall, also known as probability of detection, measures how many actual pollution events were correctly detected:

$$Recall = \frac{TP}{TP + FN} \quad (4.26)$$

The F1 score balances precision and recall for event detection performance:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (4.27)$$

Explainability will be assessed qualitatively by inspecting whether important stations, weather factors, and time windows are consistent with wind direction, distance, and pollutant history. The dashboard will be evaluated based on whether it can clearly present forecasts, AQI categories, spatial maps, explanation visualizations, and what-if scenario comparisons.

5. IMPLEMENTATION DETAILS

This chapter describes the implementation work completed by the mid-term stage and the technical plan being followed for the remaining implementation. The project is implemented as a modular software system consisting of data ingestion, preprocessing, graph construction, forecasting, explainability, backend API, and dashboard modules.

5.1. Implementation Status

The initial project phase established the problem definition, objectives, literature review, feasibility, and expected outcomes. During the mid-term phase, the work has progressed toward the technical design and implementation structure. The current implementation status is summarized in Table 5-1.

Table 5-1. Mid-Term Implementation Status

Module	Work Completed	Status
Literature and requirement analysis	Reviewed spatio-temporal forecasting, GNN, wind-aware graph, explainability, and Kathmandu Valley air quality studies	Completed
Dataset planning	Identified pollutant and meteorological variables, candidate stations, timestamp resolution, and target forecasting horizons	Completed
Preprocessing design	Defined timestamp alignment, missing-value handling, outlier treatment, scaling, and sliding-window generation	In progress
Wind-aware graph design	Formulated dynamic adjacency using distance, $PM_{2.5}$ correlation, wind direction, and wind speed	In progress
Forecasting model	Designed GAT-GRU model structure and baseline comparison plan	In progress
Dashboard	Planned station-level, map-based, time-series, AQI, and explainability views	Pending

5.2. Data Preparation Module

The data preparation module converts raw pollutant and weather records into a synchronized dataset suitable for graph-based forecasting. $PM_{2.5}$ is used as the primary target variable, while PM_{10} , temperature, humidity, wind speed, wind direction, pressure, dew point, hour of day, day of week, month, and season are considered as input features.

Records from different stations are aligned at an hourly interval. Short missing gaps are handled using interpolation, while long missing segments are removed if they would

reduce reliability. Wind direction is converted into sine and cosine components to avoid discontinuity between 359 degrees and 0 degrees. Numerical features are normalized using training-set statistics to prevent data leakage.

5.3. Graph Construction Module

The graph construction module represents each physical monitoring station as a node. The edge weight between two stations is computed using spatial and meteorological information. A higher edge weight is assigned when two stations are geographically close, historically correlated, and aligned with the wind direction. This design allows the graph to change over time as wind direction and wind speed change.

The dynamic graph at time t is represented as:

$$G_t = (V, E_t, A_t) \quad (5.1)$$

where V is the station set, E_t is the time-dependent edge set, and A_t is the weighted adjacency matrix. The graph construction output is passed to the GAT layer for spatial representation learning.

5.4. Forecasting Model Module

The primary model combines Graph Attention Network and Gated Recurrent Unit components. The GAT layer learns spatial relationships between stations, while the GRU layer learns temporal changes in pollution concentration. The model receives a sequence of graph-structured observations and predicts $PM_{2.5}$ at a future horizon.

The forecasting module will be compared with baseline models. The baseline set includes a persistence model, traditional regression models, and a GRU-only model. This comparison is necessary to verify whether the wind-aware graph and attention mechanism improve forecasting performance.

5.5. Virtual Node Module

Virtual nodes are included to estimate $PM_{2.5}$ concentration at unmonitored locations. A virtual node is connected to nearby real stations using distance and wind-aware edge weights. Since virtual nodes do not have direct sensor readings, their input features are estimated from surrounding stations and meteorological conditions.

The planned validation approach is leave-one-station-out testing. In this method, one physical station is temporarily treated as an unmonitored location, and the predicted value is compared with the actual observation from that station.

5.6. Explainability Module

The explanation module will generate three types of interpretation:

- **Spatial explanation:** GAT attention weights will show which neighboring stations influenced a target station or virtual node.
- **Feature explanation:** SHAP or similar feature attribution methods will show the contribution of PM_{2.5} history, wind speed, wind direction, humidity, temperature, and temporal features.
- **Temporal explanation:** Sensitivity across look-back windows will show whether recent hours or earlier patterns had stronger influence on the forecast.

5.7. Dashboard Module

The dashboard is planned as the user-facing component of the project. It will include station-wise PM_{2.5} forecasts, AQI categories, historical-versus-predicted plots, spatial maps, virtual-node estimates, and explanation charts. A backend API will serve processed data, model outputs, and explanation results to the frontend.

6. RESULTS AND ANALYSIS

This chapter presents the results available at the mid-term stage and analyzes the progress against the project objectives. Since the complete model training and dashboard integration are scheduled for the next phase, the current results focus on completed design outputs, dataset organization, architecture readiness, and implementation progress.

6.1. Requirement and Design Results

The first major result of the project is a clear technical requirement specification for the forecasting system. The system has been divided into data preparation, graph construction, forecasting, explainability, backend, and dashboard modules. This modular design helps reduce implementation complexity and makes evaluation easier.

The second major result is the finalized system architecture. The architecture connects pollutant data, meteorological data, wind-aware graph construction, GAT-GRU forecasting, explainability, and dashboard visualization. This design satisfies the original objective of combining spatial, temporal, wind-aware, and explainable modeling components.

6.2. Dataset Preparation Results

The required dataset structure has been identified. $PM_{2.5}$ is selected as the main forecasting target, while PM_1 , temperature, humidity, wind speed, wind direction, pressure, dew point, and temporal features are selected as input variables. The data will be organized as hourly station-wise records. A model-ready sample structure is shown in Table 6-1.

Table 6-1. Model-Ready Dataset Structure

Field	Purpose
Timestamp	Aligns all pollutant and meteorological readings at hourly resolution
Station ID and coordinates	Identifies each graph node and supports distance calculation
$PM_{2.5}$	Main prediction target and historical input feature
Meteorological variables	Captures weather influence on pollutant concentration and transport
Wind direction and wind speed	Builds wind-aware dynamic graph edges
Temporal variables	Captures daily, weekly, and seasonal patterns

6.3. Architecture Analysis

The proposed architecture is suitable for the project because it addresses the main limitations identified in the literature review. Station-wise models can learn temporal patterns but cannot fully capture pollutant movement between stations. Static graph models can capture spatial relationships but may not adapt to changing wind conditions. The proposed dynamic graph combines distance, historical correlation, wind speed, and wind-direction alignment, making it more suitable for a valley-based air quality forecasting problem.

The GAT-GRU architecture is also appropriate because GAT can learn unequal influence from neighboring stations and GRU can model temporal dependency. The attention values produced by GAT can be reused as part of the explainability module, which supports the objective of producing interpretable forecasts.

6.4. Mid-Term Progress Analysis

Table 6-2 summarizes the progress of the project at the mid-term stage.

Table 6-2. Progress Analysis at Mid-Term Stage

Work Package	Analysis	Progress
Literature review	Relevant work on GNN, GAT, dynamic graphs, explainability, and Kathmandu Valley air quality has been reviewed and linked to the project gap	Complete
Requirement analysis	Functional, non-functional, hardware, software, and feasibility requirements have been defined	Complete
System architecture	Major system blocks and data flow have been finalized	Complete
Preprocessing pipeline	Data cleaning and synchronization steps have been designed and are being implemented	Partial
Model implementation	GAT-GRU model structure and baseline comparison plan have been prepared	Partial
Evaluation and dashboard	Quantitative results, explainability outputs, virtual-node validation, and dashboard integration remain for the second phase	Pending

6.5. Expected Evaluation Outputs

The final evaluation will include MAE, RMSE, MAPE, R^2 , bias, and peak-event metrics. Results will be presented horizon-wise and station-wise. The proposed model will be

compared against baseline models to determine whether wind-aware graph construction improves forecast accuracy.

The dashboard result will be evaluated based on whether it can display forecast values, AQI categories, station maps, virtual-node estimates, and explanation outputs clearly. Since the project is a research prototype, the final analysis will focus on both prediction accuracy and interpretability.

6.6. Discussion

The project is progressing according to the mid-term expectations. The problem scope has been made specific to PM_{2.5} forecasting in Kathmandu Valley, and the system design is aligned with the project objectives. The most important remaining challenge is to convert the designed modules into tested implementation outputs and generate quantitative results. Model evaluation and dashboard integration will therefore receive the highest priority in the next phase.

7. REMAINING TASK

This chapter lists the remaining work required to complete the project after the mid-term stage. The remaining tasks focus on implementation, experimentation, analysis, dashboard integration, and final documentation.

7.1. Completed Work

The following tasks have been completed or substantially completed:

- Project topic, problem definition, objectives, scope, and applications have been finalized.
- Literature review has been prepared for air quality forecasting, graph neural networks, wind-aware graph construction, virtual nodes, and explainability.
- Functional, non-functional, hardware, and software requirements have been defined.
- System architecture and methodology have been designed.
- Dataset sources, input variables, forecasting target, and evaluation metrics have been identified.
- The wind-aware graph formulation and GAT-GRU model structure have been prepared.

7.2. Pending Work

The following tasks remain for the next phase:

1. Complete data collection and station-wise data validation.
2. Implement the preprocessing pipeline for timestamp alignment, missing-value handling, outlier treatment, normalization, and sliding-window generation.
3. Implement the wind-aware dynamic graph construction module.
4. Train and evaluate baseline models.
5. Implement, train, and tune the GAT-GRU forecasting model.
6. Validate virtual-node estimation using leave-one-station-out testing.
7. Generate explainability outputs using attention weights, feature importance, and temporal sensitivity analysis.
8. Develop backend API endpoints for forecasts, AQI categories, maps, and explanation outputs.
9. Develop and test the frontend dashboard.
10. Prepare final result tables, graphs, comparison analysis, limitations, and conclusion.

7.3. Risks and Mitigation

The major remaining risks are data gaps, model complexity, limited training time, and dashboard integration delays. These risks will be mitigated by using selected stations with usable data coverage, implementing simple baselines first, keeping the GAT-GRU model modular, using optional cloud GPU resources only when needed, and integrating the dashboard incrementally.

7.4. Remaining Milestones

Table 7-1. Remaining Project Milestones

Milestone	Expected Output
Data and preprocessing completion	Cleaned station-wise dataset and model-ready input windows
Model implementation	Baseline models and GAT-GRU forecasting model
Evaluation	Quantitative error metrics, horizon-wise comparison, and peak-event analysis
Explainability	Station influence maps, feature-importance plots, and temporal analysis
Dashboard integration	Web dashboard with forecast, AQI, spatial, and explanation views
Final documentation	Final project report, presentation, and demonstration materials

A. PROJECT BUDGET

The proposed system is mainly a software and research project. Most tools and datasets are free or open source, so the budget is focused on internet access, optional computing resources, hosting, printing, binding, and miscellaneous project expenses.

Table A-1. Estimated Project Budget

Category	Description	Estimated Cost
Internet and data access	Dataset downloads, API access, research review, and collaboration	NRs. 2,000
Cloud/GPU usage	Optional Google Colab Pro, Kaggle, or equivalent compute for model training	NRs. 5,000
Domain or hosting	Optional dashboard deployment for demonstration	NRs. 3,000
Printing and binding	Mid-term report, final report, and presentation materials	NRs. 6,000
Miscellaneous	Backup storage and unforeseen project needs	NRs. 2,000
Total Estimated Cost		NRs. 18,000

B. PROJECT TIMELINE

The project is planned in phases: literature review, dataset preparation, preprocessing, graph construction, baseline modeling, GAT-GRU implementation, explainability, dashboard development, testing, and final documentation.

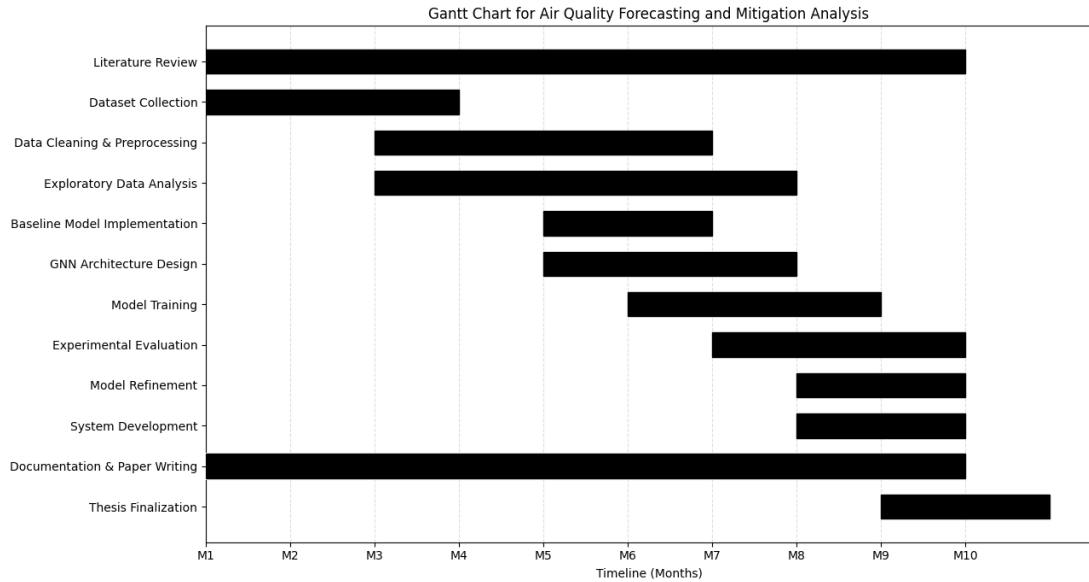


Figure B-1. Project Gantt Chart

C. DATASET DETAILS

The dataset for this project contains station-wise $\text{PM}_{2.5}$ observations and meteorological variables aligned at an hourly resolution. The major fields include timestamp, station name, station coordinates, $\text{PM}_{2.5}$, PM_1 , temperature, humidity, pressure, dew point, wind speed, wind direction, and temporal features. The final cleaned dataset will be used to generate sliding-window inputs for short-term $\text{PM}_{2.5}$ forecasting.

Table C-1. Dataset Field Description

Field	Description
Timestamp	Hourly time index used to synchronize air quality and weather records
Station metadata	Station name, station identifier, latitude, and longitude
Pollutant features	$\text{PM}_{2.5}$ target variable and available pollutant inputs such as PM_1
Meteorological features	Temperature, relative humidity, pressure, dew point, wind speed, and wind direction
Temporal features	Hour of day, day of week, month, and season for periodic behavior modeling
Missingness indicators	Optional binary indicators showing whether values were observed or imputed

REFERENCES

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017.
- [2] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [3] S. Wang, Y. Li, J. Zhang, Q. Meng, L. Meng, and F. Gao, “PM2.5-GNN: A domain knowledge enhanced graph neural network for PM2.5 forecasting,” <https://arxiv.org/abs/2002.12898>, 2020.
- [4] M. Pandey, V. Jain, N. Godhani, S. N. Tripathi, and P. Rai, “Spatio-temporal forecasting of PM2.5 via spatial-diffusion guided encoder-decoder architecture,” <https://arxiv.org/abs/2412.13935>, 2024.
- [5] M. Panja, T. Chakraborty, A. Biswas, and S. Deb, “E-STGCN: Extreme spatiotemporal graph convolutional networks for air quality forecasting,” <https://arxiv.org/abs/2411.12258>, 2024.
- [6] L. Chen, J. Xu, B. Wu, M. Lv, C. Zhan, S. Chen, and J. Chang, “HighAir: A hierarchical graph neural network-based air quality forecasting method,” <https://arxiv.org/abs/2101.04264>, 2021.
- [7] J. Xu, S. Wang, N. Ying, X. Xiao, J. Zhang, Y. Cheng, Z. Jin, and G. Zhang, “Dynamic graph neural network with adaptive edge attributes for air quality predictions,” <https://arxiv.org/abs/2302.09977>, 2023.
- [8] K. H. Hettige, J. Ji, S. Xiang, C. Long, G. Cong, and J. Wang, “AirPhyNet: Harnessing physics-guided neural networks for air quality prediction,” in *International Conference on Learning Representations*, 2024.
- [9] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNExplainer: Generating explanations for graph neural networks,” in *Advances in Neural Information Processing Systems*, 2019, pp. 9240–9251.
- [10] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.
- [11] OpenAQ, “OpenAQ API Documentation: Measurements,” <https://docs.openaq.org/resources/measurements>, 2026, accessed: 2026-06-15.

- [12] Open-Meteo, “Historical weather api documentation,” <https://open-meteo.com/en/docs/historical-weather-api>, 2026, accessed: 2026-06-15.
- [13] S. Becker, R. P. Sapkota, B. Pokharel, L. Adhikari, R. P. Pokhrel, S. Khanal, and B. Giri, “Particulate matter variability in kathmandu based on in-situ measurements, remote sensing, and reanalysis data,” NOAA / repository report, Tech. Rep., 2020.
- [14] A. K. Panday and R. G. Prinn, “Diurnal cycle of air pollution in the kathmandu valley, nepal: Observations,” *Journal of Geophysical Research: Atmospheres*, vol. 114, p. D09305, 2009.
- [15] Clean Energy Nepal, “Policy brief on air quality and health in kathmandu valley,” Clean Energy Nepal / Urban Health Initiative, Kathmandu, Nepal, Tech. Rep., n.d.
- [16] ICIMOD, “Action plan to reduce air pollution and improve air quality,” International Centre for Integrated Mountain Development, Tech. Rep., 2021.